Home Assessment: Scheduling System

Overview

You are required to design and implement a full-stack scheduling system. The system should allow users to create and manage scheduled executions of predefined tasks.

The solution must include a **React-based UI**, a **Java Spring Boot backend**, and a **database of your choice** (e.g., PostgreSQL, MySQL, etc.).

Core Requirements

1. Frontend (React)

You must build a user interface that includes:

- A **table view** displaying all existing schedules.
- Full **CRUD operations for schedulings**:
    - Create a new scheduling
    - Edit an existing scheduling
    - Delete a scheduling
- Each row in the table should represent a scheduling entity.

When creating or editing a scheduling, you must provide:

- Selection of a **task (from a predefined list)**
- Configuration of the **schedule timing**
- Input fields for **task parameters** (if applicable)

---

2. Scheduling Capabilities

You must support flexible scheduling options, similar to common calendar tools (e.g., Google Calendar), including:

- One-time execution

- Recurring schedules (e.g., every X minutes/hours)
- Weekly schedules
- Cron-like expressions (advanced scheduling)

---

3. Backend (Java + Spring Boot)

You must implement a backend service using **Spring Boot** that:

- Exposes APIs for managing schedulings (CRUD)
- Integrates with **Quartz Scheduler** for execution
- Persists all relevant data in the database
- Handles execution of scheduled jobs

---

4. Tasks (Predefined, Read-Only)

You must implement a set of **predefined tasks**. These tasks are:

- **Read-only** (cannot be created/edited via the UI)
- Selectable when creating a scheduling

Examples of tasks you should implement:

- **Log Task** – writes a message to the application log
- **Additional tasks** (e.g., dummy email sender, DB query, etc.)

---

5. Task Parameters

Each task may accept parameters.

For example:

- A log task may accept a **log message**

When creating a scheduling:

- You must allow users to **provide parameter values**
- These parameters should be passed to the task during execution

---

Bonus (Nice to Have)

- Define a **parameter schema per task**, including:
    - Parameter name
    - Type (string, number, etc.)
    - Required / optional (required flag)
- Enforce required parameters in the UI and backend validation

---

6. Testing

You are expected to include:

- **Unit tests**
- **Integration tests**
- Reasonable **test coverage**

---

7. Delivery Requirements

You must provide:

- A **Git repository** with clear structure:
    - `/frontend`
    - `/backend`
    - `/docker`
- **Docker support**:

    - Dockerfile for each component
    - A `docker-compose.yml` to run the entire system
- A **README** explaining:
    - How to run the system
    - Design decisions
    - Any assumptions made

---

8. Use of AI Tools

You may use any AI tools you choose.

However, you must:

- Clearly **state which tools you used**
- Explain **how they were used** in your solution

Evaluation Criteria

Your submission will be evaluated based on:

- **Code quality** (clean, maintainable, well-structured)
- **System design**
- **Completeness of the solution**
- **User experience (UI/UX)**
- **Test quality and coverage**